

AI Product Requirement Generator

Project Report

An AI-assisted system for turning product ideas into structured Product Requirement Documents

Project Type	AI / Product Engineering
Interface	Streamlit Web Application
LLM Layer	Groq-hosted open-source language model
AI Framework	LangChain
Retrieval	ChromaDB + sentence-transformers
Main Workflow	Business Analyst → Product Manager

Purpose of the project

The application is designed to reduce the effort involved in preparing an initial PRD. Instead of starting with a blank document, a user provides a product concept and constraints, and the system produces a structured first draft that can be reviewed and refined.

1. Problem

A product idea often begins as a short description, while a usable Product Requirement Document needs much more detail. Teams have to identify target users, describe their needs, convert those needs into user stories, define functional and non-functional requirements, and think about risks. Doing this manually takes time and can lead to inconsistent structure or missing requirements.

The aim of this project is to provide a practical starting point for that process. The user enters the product name, domain, concept, and technical or business constraints. The application then uses retrieved product-development guidance and a multi-stage AI workflow to generate a structured PRD.

2. Architecture

The system is organized as a sequential pipeline rather than a single unrestricted generation step. The Streamlit interface collects the input, the retrieval layer supplies relevant background guidance, and two agents handle different parts of the reasoning.

User	Product idea + domain + constraints
↓	
Streamlit UI	Input, navigation and results
↓	
RAG layer	Knowledge documents → ChromaDB retrieval
↓	
Business Analyst	Users, needs, stories and requirements
↓	
Product Manager	Refinement and structured PRD
↓	
Output	PRD + analytics + export options

Knowledge and retrieval. The knowledge base contains focused guidance for PRD standards, non-functional requirements, and risk analysis. These documents are embedded and stored in ChromaDB so the generation process can retrieve context relevant to the user's product idea.

Agent workflow. The Business Analyst stage concentrates on understanding the problem and users. The Product Manager stage uses that analysis to organize the final document into a consistent PRD structure. This separation makes the workflow easier to understand and evaluate.

3. Prompt Design

Prompt design was treated as part of the system architecture. The model is not simply asked to 'write a PRD'. Each agent receives a specific role, relevant input, retrieved context, and an expected output structure.

Business Analyst prompt

The first prompt frames the model as a Business Analyst. It asks the agent to interpret the product concept, identify likely users and their problems, extract useful requirements, and consider the constraints supplied by the user. Retrieved knowledge is used as supporting context rather than relying only on the model's general knowledge.

Product Manager prompt

The second prompt frames the model as a Product Manager. It receives the analysis from the previous stage and turns it into a structured PRD. The instructions emphasize clear sections, actionable requirements, realistic risks, and consistent formatting.

Prompting principles used

- Clear role assignment for each agent.
- Explicit requirements for the expected PRD structure.
- Retrieved context included where relevant.
- Separation of analysis from final document refinement.
- Consistent output formatting for display and export.

4. Evaluation

Evaluation focused on whether the complete application workflow works as intended, rather than judging a single model response in isolation. The project includes end-to-end tests covering the main application pipeline. During development, the workflow was checked from user input through retrieval, agent execution, PRD generation, result display, analytics, and exports.

Evaluation areas

- **Input handling:** product information and constraints are accepted through the Streamlit interface.
- **RAG:** the knowledge base can provide supporting context to the generation workflow.
- **Agents:** the Business Analyst and Product Manager stages execute in the intended order.
- **Output:** the generated PRD contains structured sections such as personas, user stories, functional requirements, non-functional requirements, and risks.
- **Interface:** results, navigation, analytics, and export options remain usable after generation.

The application was also tested after deployment on Streamlit Community Cloud. The final version supports cloud secrets while retaining local environment-variable support for development.

5. Challenges

One of the main challenges was deciding what information should be placed in the knowledge base. A large collection of unrelated documents could make retrieval less useful, so the project uses focused documents covering PRD standards, non-functional requirements, and risk guidance.

Another challenge was coordinating the two agents. If the first stage produced an unstructured or incomplete analysis, the second stage could not reliably turn it into a useful PRD. Role-specific prompts and a defined handoff between the agents were therefore important.

Deployment also required separate handling of local and cloud configuration. The application was adjusted so that the LLM credential can be supplied through Streamlit Secrets in the deployed version without exposing the secret in the repository.

Finally, the interface needed to remain simple despite having several functions. Navigation, results, analytics, exports, and the sidebar controls were kept within the same application so that a user can complete the workflow without moving between multiple tools.

6. Future Enhancements

The current version is intended as a strong first version of an AI-assisted PRD workflow. The following improvements could extend its usefulness:

- Add specialized knowledge bases for domains such as healthcare, fintech, education, and e-commerce.
- Introduce additional agents for security, UX, technical architecture, or compliance review.
- Allow users to save, reopen, compare, and revise previous PRDs.
- Add richer document exports, including professionally formatted PDF and DOCX output.
- Add quantitative evaluation for requirement coverage, consistency, and retrieval quality.
- Provide controls for selecting which PRD sections should be generated or regenerated.

Conclusion

The AI Product Requirement Generator demonstrates a practical combination of retrieval-augmented generation, prompt engineering, and multi-agent workflow design. It takes a relatively short product description and turns it into a structured PRD that can be reviewed, analyzed, and exported. The architecture also leaves room for future domain knowledge, additional agents, and stronger evaluation methods.

Project deliverables: source code, architecture documentation, live application, demo video, and this project report.